

NutriMinds Specialist Verification Plugin

Generated: 2026-06-25

Purpose

The NutriMinds Specialist Verification plugin provides the local WordPress frontend for gut health specialist registration and verification intake.

The current implementation stores specialist applications locally in WordPress, prepares the multilingual interface, and keeps the future backend/API integration separated until the external platform details are finalized.

Current Status

- Plugin name: NutriMinds Specialist Verification
- Plugin folder: wp-content/plugins/nutriminds-doctor-verification
- Shortcode: [nutriminds_registration]
- Active page: Specialist Registration
- Local URL: http://localhost:8088/specialist-registration/
- Supported languages: English and German
- Backend/API status: staging Ocelot Signup integration available after admin approval
- File upload storage status: uploaded through WordPress Media Library and attached to the application
- Admin approval status: pending/approved/rejected review flow implemented
- Rejection email status: admins send a polite no-reply notification when rejecting an application
- Platform settings page: NutriMinds > Settings
- Secure data endpoint: /wp-json/nutriminds/v1/application

The internal plugin folder still uses the earlier "doctor verification" slug to avoid breaking the active WordPress plugin reference. Public-facing text now uses "specialist".

Main Files

`nutriminds-doctor-verification.php`

This is the main WordPress plugin file. It registers the shortcode, loads frontend assets, manages language selection, stores frontend submissions, registers the admin review screens, exposes translated strings to JavaScript, and injects the language flags into the first block navigation menu.

Important responsibilities:

- Registers the `[nutriminds_registration]` shortcode.
- Enqueues `assets/css/registration.css` globally.
- Registers and enqueues `assets/js/registration.js` only when the form shortcode renders.
- Loads translations from `languages/en.json` and `languages/de.json`.
- Reads the selected language from the `nm_lang` query parameter or the `nutriminds_lang` cookie.
- Falls back to the WordPress locale, then English.
- Appends the English and German flag switcher to the main navigation.
- Registers the `nm_specialist_app` custom post type.
- Handles frontend AJAX submissions through `admin-ajax.php`.
- Stores pending applications, uploaded documents, selected professions, and review status.
- Adds the **NutriMinds > Applications** admin review queue.
- Adds the **NutriMinds > Settings** page for the staging GraphQL endpoint, API token, invite code, and connection test.
- Sends approved applications to the staging Ocelot Signup mutation when platform integration is enabled.
- Sends a polite rejection notification email when an admin rejects an application.
- Registers a protected WordPress REST data endpoint for Ocelot to request approved specialist data.

templates/registration-form.php

This file renders the four-step registration form. All visible text is now loaded through the plugin translation helper:

```
$this->t('translation.key')
```

The form currently contains:

- Step 1: specialist personal details
- Step 2: profession and specialist field selection
- Step 3: document upload controls
- Step 4: review and consent

assets/js/registration.js

This file powers the frontend interaction:

- Step navigation
- Profession filtering
- Profession selection
- Category filter chips and collapsible profession groups to keep the long specialist list manageable
- Selected profession summary
- Review summary
- Inline validation before moving between steps or submitting
- Top-level validation summary when a step has errors
- AJAX submission into the local WordPress admin queue

JavaScript text and profession data are passed from PHP using:

```
window.NutriMindsRegistration
```

The template also prints a `data-nm-registration-config` JSON block so the frontend can reliably read the larger profession lists.

Validation currently checks required text fields, email format, phone format, required profession selection, required consent checkboxes, required document uploads, file type, and a maximum file size of 10 MB. The backend repeats the phone, required-field, file-size, and allowed-file-type checks before storing the application.

assets/css/registration.css

This file contains the visual style for the specialist verification form and language switcher. The style was adjusted toward the NutriMinds design direction:

- clean health/wellness layout
- NutriMinds blue, coral, sand, and neutral palette
- bundled NutriMinds logo in the form header, with WordPress custom logo preferred when available
- form shell widened to 1080px for the profession-selection step
- header logo constrained to `max-height: 20px` with automatic width
- rounded form controls
- step-based flow
- specialist-oriented wording
- menu flag styling
- Dosis font loaded through WordPress and applied globally across the public website with `font-family: 'Dosis', Sans-serif;`

Document-Based Verification

The form no longer collects a typed professional-license number. Instead, verification is based on uploaded documents, because the professions in the list do not share a single universal identifier and the safest review path is document-based.

Required uploads:

- Professional license or registration document. Germany examples include Approbation, Berufserlaubnis, LANR/EFN evidence where relevant, chamber membership evidence, or Heilpraktiker-Erlaubnis.
- Diploma, certificate, or specialist field credential.
- Identity document copy, such as ID, driving license, or passport.

The address field in the personal-details step is optional and is stored only to help admins match documents when needed.

Research notes:

- German physicians need Approbation or a temporary permit to practise medicine under the Berufsordnung/Bundesärzteordnung framework.
- Psychotherapists need Approbation or a temporary permit to use the protected professional title.
- Heilpraktiker need an official permit to practise medicine without being licensed as a physician.
- LANR is physician-specific and not a universal identifier for all NutriMinds professions.

languages/en.json and languages/de.json

These JSON files are the plugin language files. All visible strings should be added here first, then referenced by key in PHP or JavaScript.

Examples:

```
"form.title": "Specialist verification"
"button.continue": "Continue"
"js.frontendComplete": "Frontend prototype complete..."
```

German translations use standard German characters and are stored directly in UTF-8 JSON.

The profession list is stored under `specialtyGroups` in each language file. The English list was imported from `nutriminds_en.html` and currently contains 8 groups with 71 entries. The German list was imported from `nutriminds_de.html` and currently contains 19 groups with 143 entries.

Language System

The plugin supports two languages:

- English: en
- German: de

The user can switch language from the main menu flag links:

- UK flag: English
- German flag: Deutsch

The switcher adds a query parameter:

```
?nm_lang=en  
?nm_lang=de
```

When a supported language is selected, the plugin stores it in this cookie:

```
nutriminds_lang
```

The selected language is then reused on future visits. If no cookie exists, the plugin checks the WordPress locale and uses German when the locale starts with de. Otherwise it uses English.

How To Add A New Text

1. Add the same key to both JSON files:

```
languages/en.json  
languages/de.json
```

2. In PHP templates, print the text with:

```
<?php echo esc_html($this->t('your.key')); ?>
```

3. For input placeholders or attributes, use:

```
<?php echo esc_attr($this->t('your.key')); ?>
```

4. For JavaScript text, add the key to `get_client_config()` in the main plugin file, then read it in `registration.js` using the label helper.

How To Add A New Language

1. Create a new language file:

```
languages/fr.json
```

2. Add the language code to `is_supported_language()`.

3. Add the flag and label in `render_language_switcher()`.

4. Add locale detection in `get_current_language()` if automatic detection is needed.

5. Add translated profession groups and entries under `specialtyGroups`.

How To Use The Form On A Page

Add this shortcode to any WordPress page:

```
[nutriminds_registration]
```

The plugin will render the registration form and load the required JavaScript configuration for the current language.

Current Form Fields

The current frontend form collects:

- first name
- last name
- email
- phone
- optional address
- selected profession and specialist fields
- registration document upload control
- credential document upload control
- ID, driving license, or passport upload control
- terms consent
- platform account invitation consent

Submissions are saved locally as WordPress `nm_specialist_app` records. The external platform is only called later, after an admin approves the application.

Admin Review Flow

The first local backend/admin slice is implemented.

- Frontend submissions create a `nm_specialist_app` custom post.
- The initial application status is stored as `pending`.
- Uploaded registration, credential, and identity documents are stored as WordPress attachments connected to the application.
- Admins can open `NutriMinds > Applications` in the dashboard.
- The admin queue has `Pending`, `Approved`, and `Rejected` tabs.
- Pending applications can be approved or rejected locally.
- When an application is approved, the plugin attempts to send the applicant email to the configured staging Ocelot Signup mutation.
- The local approval is kept even if the staging API fails.
- When an application is rejected, WordPress sends a polite notification email to the applicant.
- Rejection emails use the applicant's saved language, with text stored in `languages/en.json` and `languages/de.json`.
- Rejection emails are sent as plain text from the WordPress admin email with the display sender `no-reply`.
- Approved applications are available from `NutriMinds > Application Records` and the custom post edit/detail screen.
- The detail screen shows applicant fields, selected professions, primary profession, uploaded documents, language, submitted date, local status, platform status, last sync time, last platform error, and rejection email status.
- Approved rows include a `Retry platform` action so the Signup request can be sent again after configuration or API errors are fixed.

Local Email Testing With Mailpit

Local WordPress emails are captured with Mailpit instead of being sent to real recipients.

Mailpit runs in Docker:

```
docker run -d --name nutriminds-mailpit -p 127.0.0.1:8025:8025 -p 127.0.0.1:1025:1025 axllent/mailpit
```

Mailpit URLs:

```
SMTP: 127.0.0.1:1025
Inbox: http://localhost:8025/
```

The plugin configures WordPress mail through the `phpmailer_init` hook when Mailpit is enabled or when the site URL host is `localhost` or `127.0.0.1`.

The local sender is:

```
no-reply <WordPress admin email>
```

Useful Docker commands:

```
docker ps --filter name=nutriminds-mailpit
docker start nutrिमinds-mailpit
docker stop nutrिमinds-mailpit
```

The Mailpit option is also visible in:

```
WordPress Admin > NutriMinds > Settings > Local email testing
```

Platform Settings

Open the settings screen here:

```
WordPress Admin > NutriMinds > Settings
```

The active platform integration currently uses:

```
https://os.nutriminds.net/api
```

Previous staging endpoints kept for rollback:

```
https://os.nutriminds.net.stage.ocelot-social.it4c.org/api
https://stage.ocelot.social/api
```

The settings screen contains:

- Enable integration

- GraphQL endpoint
- API token
- Optional invite code
- Save and test connection button
- Inbound data endpoint URL
- Inbound data endpoint token
- Generate inbound token button

The API token is stored server-side and is never printed back in the settings form. Leave the token field empty when saving if the existing token should remain unchanged.

For a safer deployment configuration, the token can also be defined in `wp-config.php`:

```
define('NUTRIMINDS_PLATFORM_TOKEN', 'your-token-here');
```

When that constant exists, it is used instead of the token saved in WordPress settings.

Secure Data Endpoint

The second integration direction is Ocelot calling WordPress after the signup link flow starts or after the user confirms the signup.

Endpoint:

```
POST /wp-json/nutriminds/v1/application
```

Full local URL example:

```
http://localhost:8088/wp-json/nutriminds/v1/application
```

Production URL example:

```
https://nutriminds.net/wp-json/nutriminds/v1/application
```

Authentication:

```
Authorization: Bearer <inbound-data-endpoint-token>
```

The endpoint token is separate from the Ocelot API token used by WordPress. Generate or paste it in:

WordPress Admin > NutriMinds > Settings > Inbound data endpoint

Request body:

```
{
  "email": "approved-specialist@example.com"
}
```

Successful response:

```
{
  "email": "approved-specialist@example.com",
  "firstName": "Example",
  "lastName": "Specialist",
  "displayName": "Example Specialist",
  "locale": "en",
  "verification": {
    "status": "approved",
    "source": "nutriminds",
    "approvedAt": "2026-06-29 12:00:00",
    "externalReference": "nutriminds-wp-123"
  },
  "primarySpecialty": {
    "id": "gut-health-specialist",
    "name": "Gut Health Specialist",
    "category": "Gut Health",
    "tags": []
  },
  "specialties": []
}
```

Security behavior:

- The endpoint only accepts POST requests.
- The applicant email must be sent in the JSON body, not in the URL query string.
- A bearer token is required.
- Only approved applications are returned.
- Pending or rejected applications return 404.
- Missing or invalid tokens return 401.
- If the endpoint is disabled or has no token configured, it returns 404.

- Basic rate limiting is applied per remote IP and email.
- Uploaded document URLs, phone numbers, and raw internal review data are not exposed.
- Each successful data request updates the application metadata field `data_endpoint_last_accessed_at`.

Planned Backend Flow

Implemented backend responsibilities:

- Trigger the external platform Signup mutation only after approval.
- Store API status, timestamps, response JSON, and error messages.
- Allow retrying the platform Signup call from the admin queue.
- Provide a secure approved-application data endpoint for Ocelot prefill, badges, and verification.
- Send rejection notification emails to applicants.

Expected future backend responsibilities:

- Decide whether SignupVerification remains fully platform-side.

External Platform API Notes

Based on the platform documentation reviewed earlier, WordPress uses Signup after admin approval:

```
Signup(email: String!, locale: String!, inviteCode: String = null):  
EmailAddress
```

Current mutation body:

```
mutation Signup($email: String!, $locale: String!, $inviteCode: String)  
{  
  Signup(email: $email, locale: $locale, inviteCode: $inviteCode) {  
    createdAt  
    email  
    verifiedAt  
  }  
}
```

The variables are mapped like this:

- email: application email address

- locale: saved application language, currently `en` or `de`
- inviteCode: optional setting value, or null when empty

The API request sends the token in this HTTP header:

```
Authorization: Bearer <token>
```

The SignupVerification mutation should remain platform-side unless the external developer confirms a different responsibility split:

```
SignupVerification(...): User
```

Before production work starts, the following details are still needed:

- inviteCode rules
- locale mapping rules
- duplicate email behavior
- production endpoint and token

Verification Commands Used

The following checks were run locally:

```
php -l nutriminds-doctor-verification.php
php -l templates/registration-form.php
node --check assets/js/registration.js
php JSON decode check for languages/en.json and languages/de.json
curl staging GraphQL endpoint with query { __typename }
```

The local page was also checked with:

```
http://localhost:8088/specialist-registration/?nm_lang=en
http://localhost:8088/specialist-registration/?nm_lang=de
```

Both languages render the translated form and the main menu language flags.

Developer Notes

- Keep all future visible strings in the JSON language files.

- Do not hardcode English text in PHP templates or JavaScript.
- Keep backend/API behavior separate from frontend rendering.
- Do not call the external platform before admin approval.
- Keep user-facing wording as "specialist".
- Rename the internal plugin slug only in a controlled migration, because changing the plugin folder can deactivate or disconnect the current plugin installation.

----- Api key ----- oak_9rmjKptSe5ru8fdFVuYOUquLe4-rKNrDjOG5l169AtE